

Solutionnaire du TP3

À votre tour

0. Importer

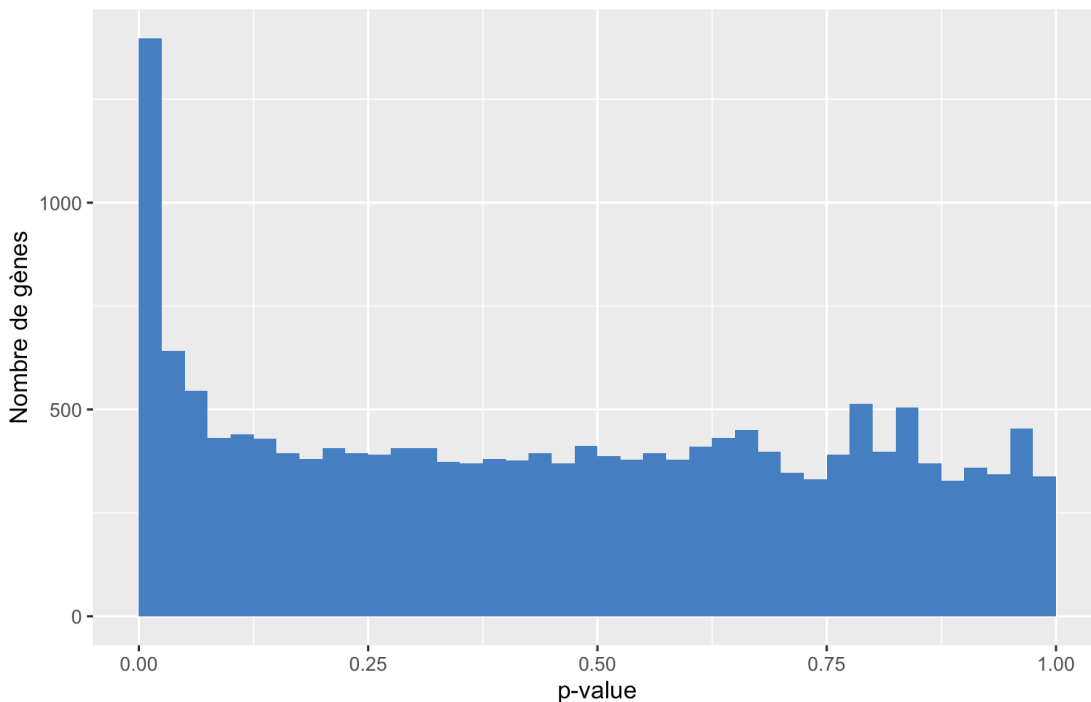
```
library(tidyverse)
library(ggrepel)
library(pheatmap)
de <- read_csv("/home/public/visu/de_resultats.csv")
tpm <- read_csv("/home/public/visu/tpm.csv")
```

```
nrow(de)
#> [1] 19320
sum(is.na(de$padj))
#> [1] 6308
sum(de$padj < 0.05, na.rm = TRUE)
#> [1] 361
```

19320 gènes, dont 6308 **non testés** (`padj` manquante : trop peu exprimés), et 361 significatifs au seuil de 5 % (FDR). Sans `na.rm = TRUE`, la somme vaudrait **NA**.

1. L'histogramme des p-values

```
ggplot(de, aes(pvalue)) +
  geom_histogram(binwidth = 0.025, boundary = 0, fill = "#4A7FC1") +
  labs(x = "p-value", y = "Nombre de gènes")
#> Warning: Removed 2073 rows containing non-finite outside the scale range
#> (`stat_bin()`).
```



Un fond à peu près **plat** (les gènes qui ne changent pas : p-values uniformes) et un **pic près de 0** (les gènes qui changent vraiment). C'est la forme attendue d'une analyse saine : un histogramme en U, ou un pic près de 1, signalerait un problème de modèle. Les 2073 **NA** de **pvalue** sont retirés, avec un *warning*.

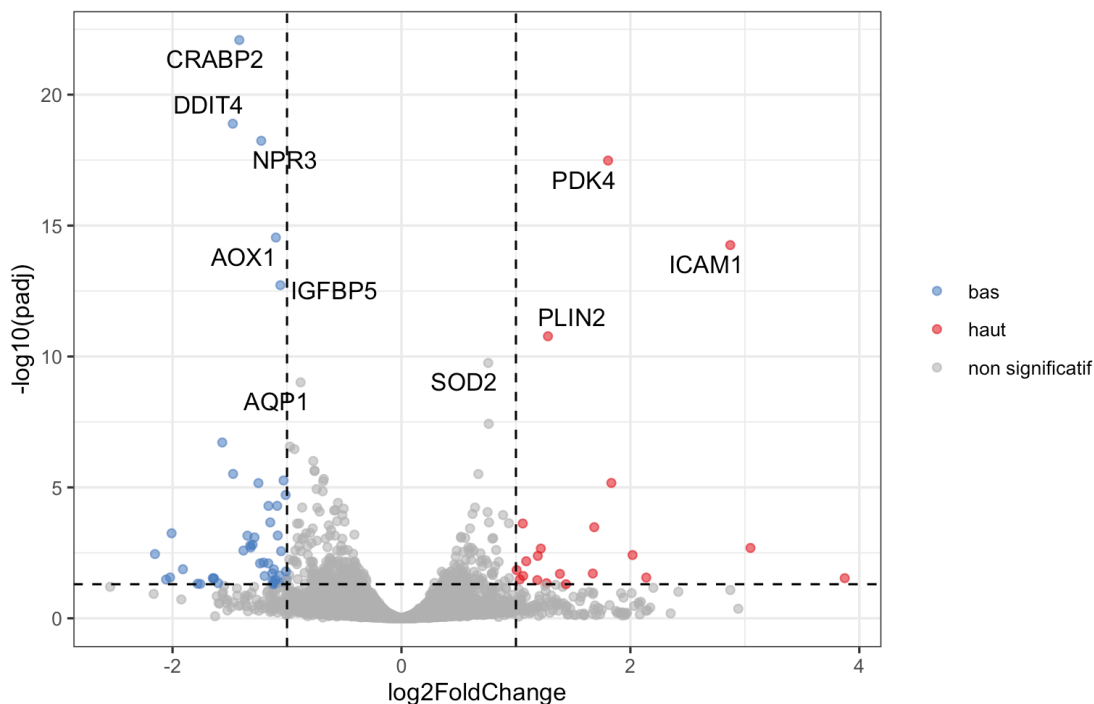
2. et 3. Le volcano plot

```
de_v <- de |>
  filter(!is.na(padj)) |>
  mutate(statut = case_when(padj < 0.05 & log2FoldChange > 1 ~ "haut",
    padj < 0.05 & log2FoldChange < -1 ~ "bas",
    .default = "non significatif"))

count(de_v, statut)
#> # A tibble: 3 × 2
#>   statut      n
#>   <chr>    <int>
#> 1 bas         47
#> 2 haut        21
#> 3 non significatif 12944

top <- de_v |> slice_min(padj, n = 10)

ggplot(de_v, aes(x = log2FoldChange, y = -log10(padj), color = statut)) +
  geom_point(alpha = 0.6) +
  geom_vline(xintercept = c(-1, 1), linetype = 2) +
  geom_hline(yintercept = -log10(0.05), linetype = 2) +
  geom_text_repel(data = top, aes(label = gene), color = "black") +
  scale_color_manual(values = c(haut = "#E30513", bas = "#4A7FC1", `non significatif` = '
  labs(color = NULL) +
  theme_bw()
```



Seuls 68 gènes passent les **deux** seuils : la plupart des gènes significatifs changent de moins d'un facteur 2 ($|\log_2FC| < 1$). Les 10 gènes les plus significatifs ne sont pas forcément ceux qui changent le plus : **top** est choisi sur **padj**, pas sur **log2FoldChange**.

4. Le sens du log2FoldChange

```
filter(de, gene == "ICAM1")
#> # A tibble: 1 × 5
#>   gene baseMean log2FoldChange pvalue padj
#>   <chr>   <dbl>         <dbl>   <dbl> <dbl>
#> 1 ICAM1     79.8           2.87 2.54e-18 5.56e-15
filter(tpm, gene == "ICAM1")
#> # A tibble: 1 × 7
#>   gene cond1_1 cond1_2 cond1_3 cond2_1 cond2_2 cond2_3
#>   <chr>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1 ICAM1     3.48     3.79     4.55     0.432     0.64     0.506
```

ICAM1 a un log2FoldChange de +2.87, et il est **plus exprimé dans cond1** (environ 4 TPM contre 0.5). Un log2FoldChange positif signifie donc ici « plus exprimé dans **cond1** que dans **cond2** » : **cond2** est la référence. Le nom des colonnes ne le dit pas : seule la vérification sur les données (ou l'en-tête des résultats de DESeq2, module 9) permet d'écrire « induit » ou « réprimé » sans se tromper de sens.

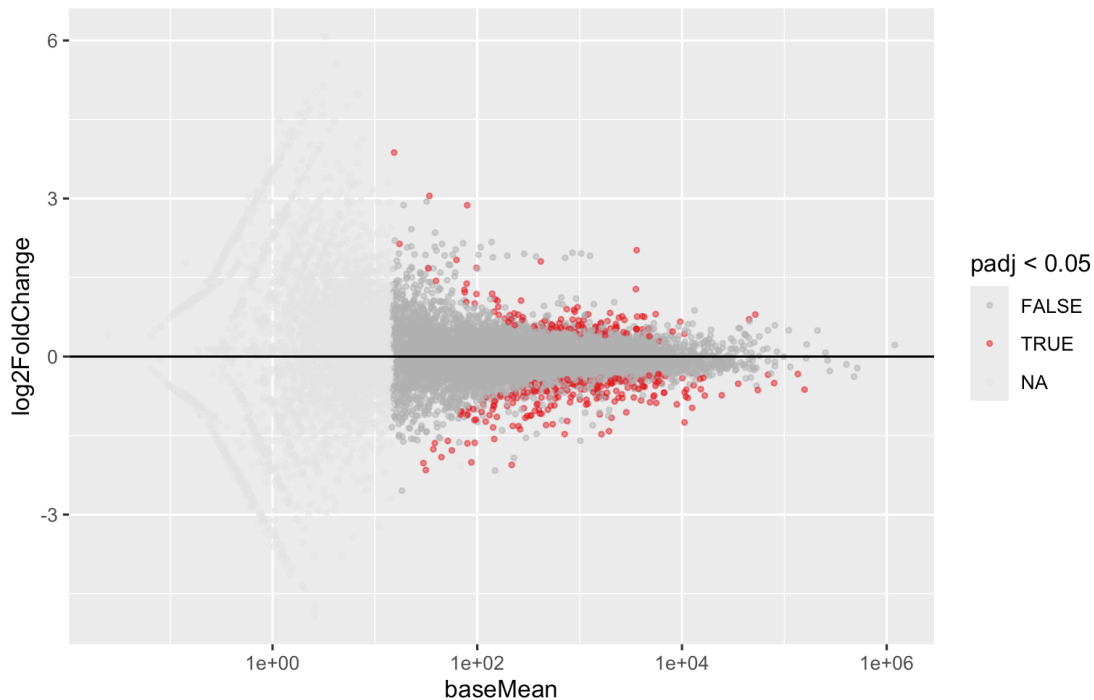
5. Le MA plot

```
ggplot(de, aes(x = baseMean, y = log2FoldChange, color = padj < 0.05)) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_x_log10() +
  geom_hline(yintercept = 0) +
```

```

scale_color_manual(values = c(`TRUE` = "#E30513", `FALSE` = "grey70"), na.value = "grey")
#> Warning in scale_x_log10(): log-10 transformation introduced infinite values.
#> Warning: Removed 2071 rows containing missing values or values outside the scale range
#> (`geom_point()`).

```



Les gènes peu exprimés (à gauche) ont des log2FoldChange très dispersés mais rarement significatifs : avec peu de lectures, un grand changement peut être dû au hasard. Les gènes à `baseMean = 0` disparaissent (`log10(0) = -Inf`), avec un *warning*.

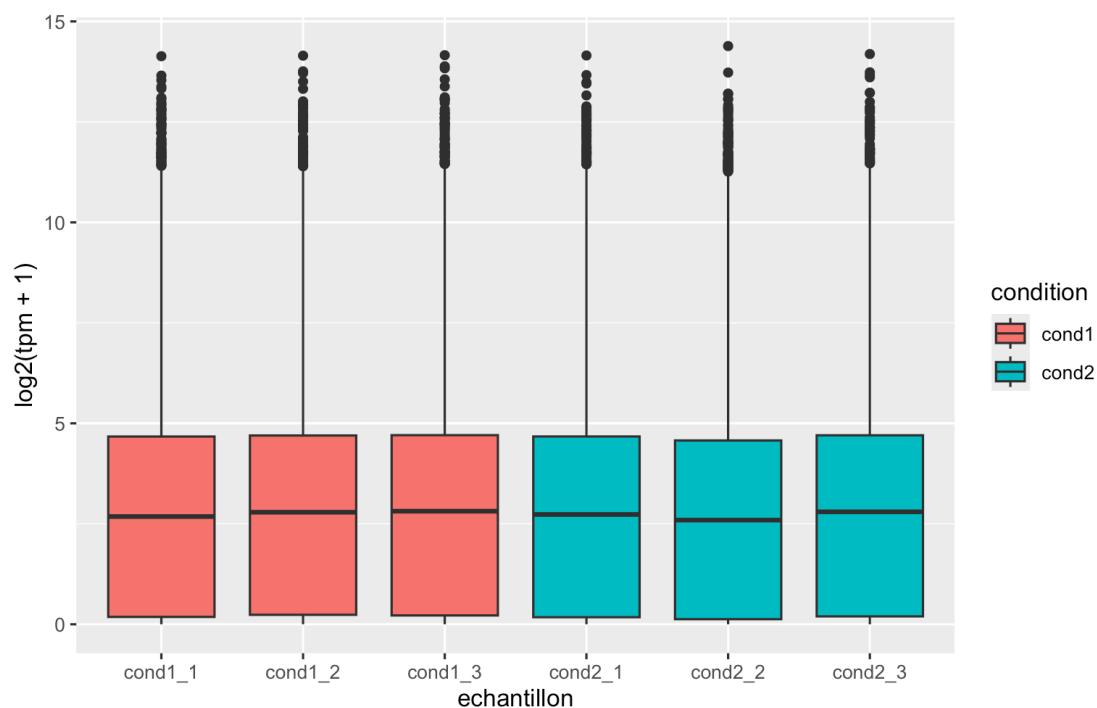
6. Les distributions par échantillon

```

tpm_long <- tpm |> pivot_longer(~gene, names_to = "echantillon", values_to = "tpm")

ggplot(tpm_long, aes(x = echantillon, y = log2(tpm + 1), fill = str_sub(echantillon, 1, 5))) +
  geom_boxplot() +
  labs(fill = "condition")

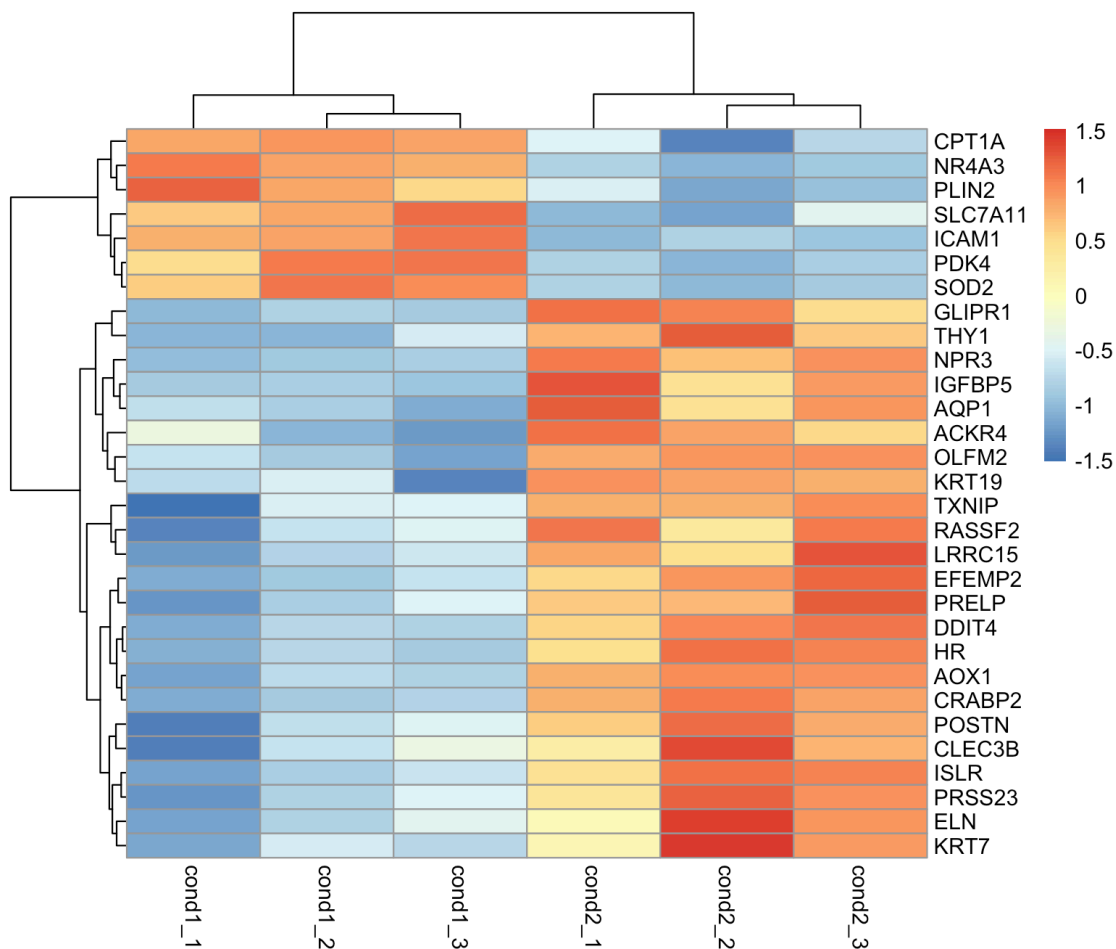
```



Les six distributions sont très proches : les échantillons sont **comparables** (même profondeur, même normalisation). Un échantillon décalé serait suspect.

7. La heatmap

```
genes <- de |> slice_min(padj, n = 30) |> pull(gene)
m <- tpm |>
  filter(gene %in% genes) |>
  column_to_rownames("gene") |>
  as.matrix()
pheatmap(log2(m + 1), scale = "row")
```



Les échantillons se séparent en deux groupes, **un par condition** : c'est attendu, puisqu'on a choisi les gènes qui différencient les conditions. Une heatmap de gènes sélectionnés ainsi ne « prouve » donc pas que les conditions sont différentes.

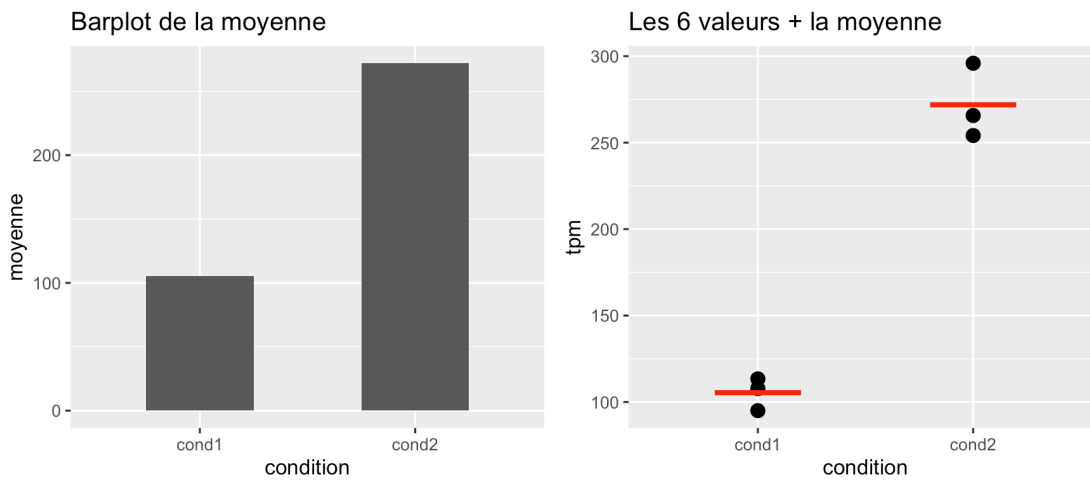
8. CRABP2 : barre ou points ?

```
crabp2 <- tpm_long |>
  filter(gene == "CRABP2") |>
  mutate(condition = str_sub(echantillon, 1, 5))

p_barre <- crabp2 |>
  summarise(moyenne = mean(tpm), .by = condition) |>
  ggplot(aes(condition, moyenne)) + geom_col(width = 0.5) + ggtitle("Barplot de la moyenne")

p_points <- ggplot(crabp2, aes(condition, tpm)) +
  geom_point(size = 3) +
  stat_summary(fun = mean, geom = "crossbar", width = 0.4, color = "red") +
  ggtitle("Les 6 valeurs + la moyenne")

gridExtra::grid.arrange(p_barre, p_points, ncol = 2)
```



Les **points** : on voit les 3 réplicats, leur dispersion, et que les deux groupes ne se chevauchent pas. Le barplot cache tout cela, et commence à 0 alors que la question porte sur l'écart entre ~100 et ~270.

Et si... ?

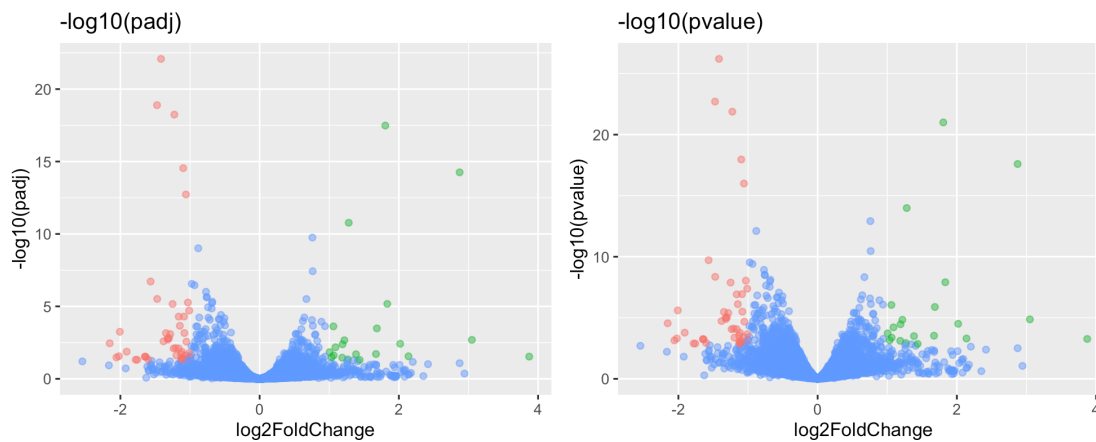
1. `if_else()` avec les **NA**

```
de |>
  mutate(statut = if_else(padj < 0.05, "signif", "non")) |>
  count(statut)
#> # A tibble: 3 × 2
#>   statut      n
#>   <chr> <int>
#> 1 non    12651
#> 2 signif  361
#> 3 <NA>   6308
```

Trois catégories : `if_else()` renvoie **NA** quand `padj` est **NA**, et `ggplot2` en fait une catégorie grise « NA » dans la légende. Ces points ne sont d'ailleurs pas tracés sur le volcano plot ($y = -\log_{10}(\text{NA})$), avec un *warning*. Il faut décider : retirer les gènes non testés, ou les classer explicitement.

2. `pvalue` ou `padj` en y

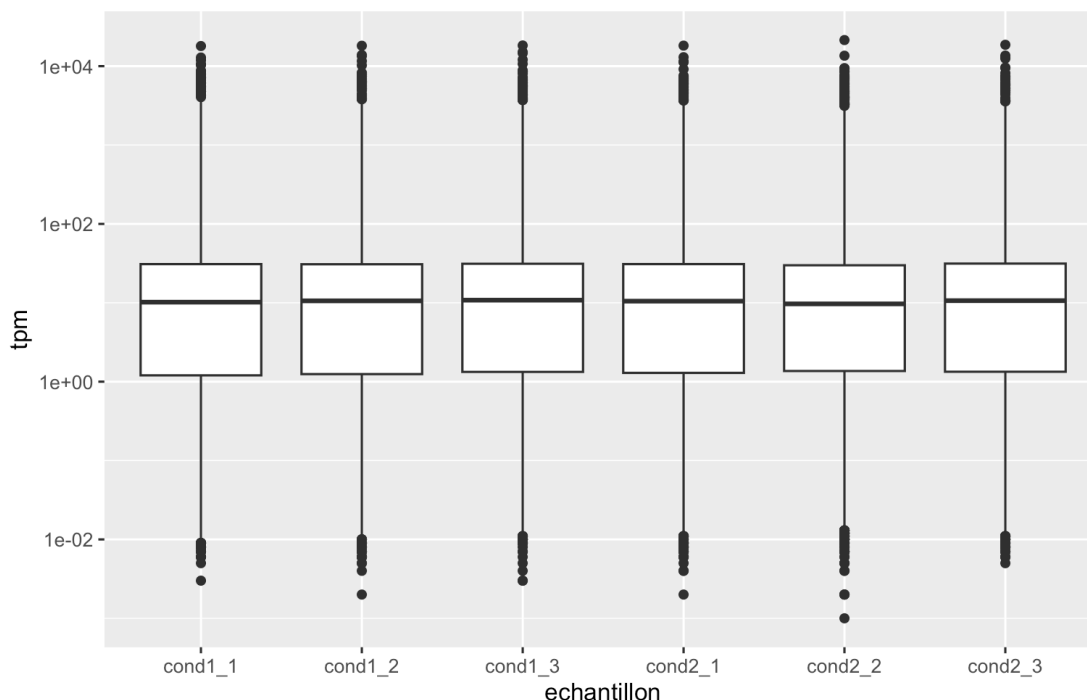
```
g1 <- ggplot(de_v, aes(log2FoldChange, -log10(padj), color = statut)) + geom_point(alpha
  theme(legend.position = "none") + ggtitle("-log10(padj)")
g2 <- ggplot(de_v, aes(log2FoldChange, -log10(pvalue), color = statut)) + geom_point(alpha
  theme(legend.position = "none") + ggtitle("-log10(pvalue)")
gridExtra::grid.arrange(g1, g2, ncol = 2)
```



La forme est presque la même (`padj` est une transformation croissante de `pvalue`), mais **plus haute** avec `pvalue`. Les couleurs ne changent pas si elles sont calculées sur `padj`. En revanche, une ligne horizontale à `-log10(0.05)` sur l'axe `pvalue` ne correspond plus au seuil utilisé pour colorer : beaucoup de points gris passeraient au-dessus. Axe et seuil doivent utiliser la même colonne.

3. `scale_y_log10()` avec des zéros

```
ggplot(tpm_long, aes(echantillon, tpm)) +
  geom_boxplot() +
  scale_y_log10()
#> Warning in scale_y_log10(): log-10 transformation introduced infinite values.
#> Warning: Removed 20297 rows containing non-finite outside the scale range
#> (`stat_boxplot()`).
```

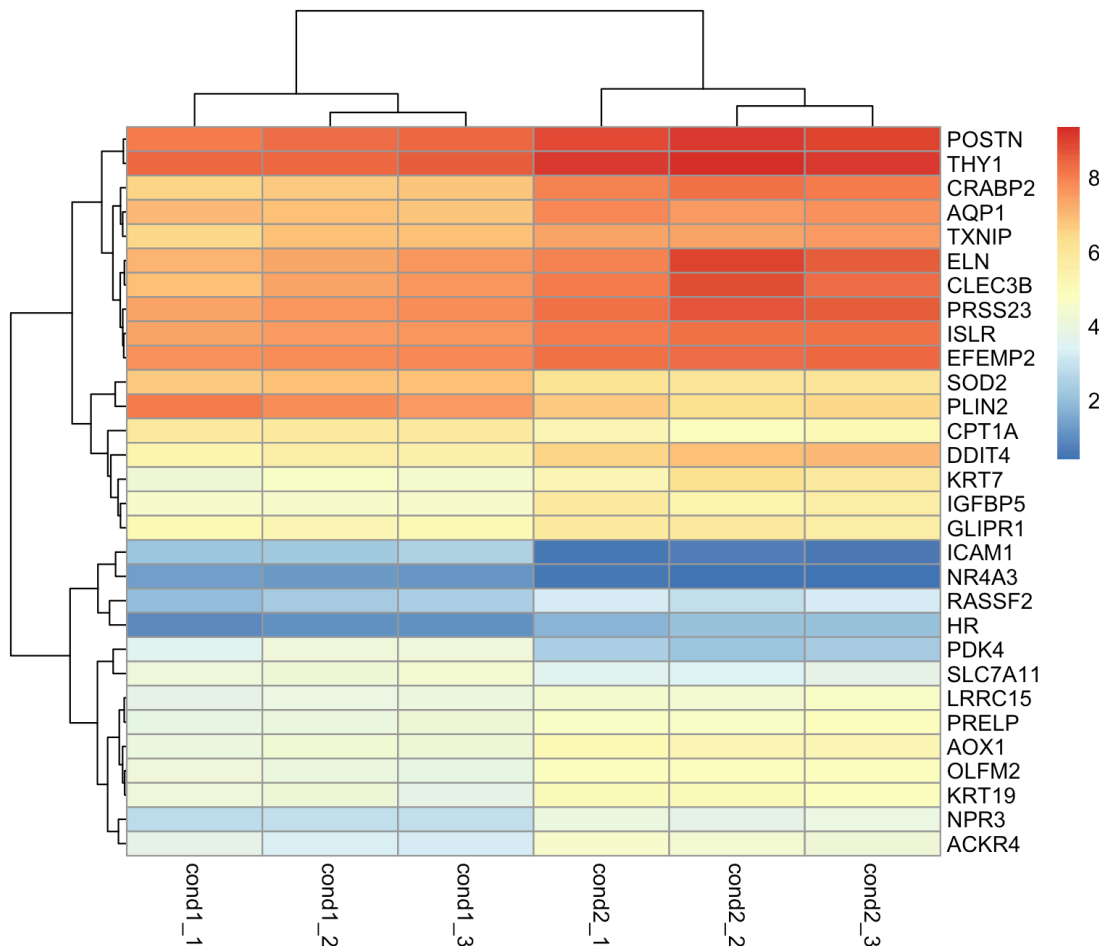


`log10(0) = -Inf` : les 20297 valeurs nulles sont **retirées** avant le calcul des boxplots (un *warning* le dit). Les médianes affichées (~10 TPM) sont celles des gènes **exprimés** seulement, deux fois plus hautes que les vraies

médianes (~5.5 TPM). `log2(tpm + 1)` garde tous les gènes : un 0 devient 0.

4. Sans `scale = "row"`

```
pheatmap(log2(m + 1))
```



La couleur montre surtout **le niveau d'expression** de chaque gène : quelques gènes très exprimés sont rouges partout, d'autres bleus partout. Les différences **entre conditions**, qui sont la question, sont écrasées. `scale = "row"` centre et réduit chaque gène : on compare chaque échantillon à la moyenne **de ce gène**.

5. Rouge et vert

Environ 8 % des hommes (et 0.5 % des femmes) ont une forme de daltonisme rouge-vert : pour eux, les gènes « haut » et « bas » se confondent. Rouge et bleu, ou une palette comme `viridis` ou `scale_color_brewer(palette = "Set1")`, restent lisibles. On peut aussi doubler la couleur par la **position** (gauche/droite) ou la **forme** des points.